Cahier d'exercices · Créer un jeu 2D avec Godot

Jeux vidéo · 4 exercices et leurs corrigés en GDScript

Ce cahier accompagne la formation « Créer un jeu 2D avec Godot » (Godot 4). Chaque exercice ajoute une fonctionnalité à ton jeu. Dans les codes, l'indentation se fait avec la touche Tab, comme dans l'éditeur de Godot.

Conseil : fais les exercices dans l'ordre, sans regarder les corrigés, puis compare. Coche les critères de réussite : ils sont aussi la grille utilisée pour évaluer un projet.

Exercices

Exercice 1 Les vies du joueur

Débutant · 15 min

Dans un script, crée une variable vies := 3 et une fonction perdre_une_vie() qui retire une vie, affiche le nombre restant et affiche « Partie terminée » à 0. Appelle-la 3 fois dans _ready().

Critères de réussite :

- ☐ Le message final s'affiche exactement une fois.
- ☐ Les vies ne descendent jamais sous 0.

Exercice 2 Courir avec Maj

Intermédiaire · 20 min

Ajoute une action « courir » dans Projet > Paramètres du projet > Contrôles, associée à la touche Maj. Quand elle est enfoncée, le joueur va deux fois plus vite.

Critères de réussite :

- ☐ L'action est créée dans les Contrôles (InputMap), pas testée par code de touche.
- ☐ La vitesse normale est réglable avec @export.

Exercice 3 Le minuteur de 60 secondes

Intermédiaire · 25 min

Ajoute un nœud Timer (Wait Time 60, One Shot, Autostart) et un Label. Le Label affiche le temps restant en secondes ; à la fin, il affiche « Temps écoulé ! » et le joueur ne peut plus bouger.

Critères de réussite :

- ☐ Le temps restant est lu avec time_left.
- ☐ Le signal timeout est connecté.

Exercice 4 Un ennemi qui patrouille

Intermédiaire · 35 min

Crée un ennemi (Area2D) qui fait des allers-retours horizontaux sur 200 pixels. S'il touche le joueur, celui-ci perd une vie.

Critères de réussite :

- ☐ Le mouvement utilise delta pour être régulier sur tous les ordinateurs.
- ☐ Le contact passe par le signal body_entered.

 · Licence CC BY 4.0 : réutilisation libre avec mention « TOB-MAQ ».

Corrigés et éléments de réponse

Exercice 1 · Les vies du joueur

max(vies - 1, 0) empêche d'avoir des vies négatives même si la fonction est appelée trop souvent.

```
extends Node2D

var vies := 3

func _ready():
    for i in 3:
        perdre_une_vie()

func perdre_une_vie():
    vies = max(vies - 1, 0)
    print("Vies : ", vies)
    if vies == 0:
        print("Partie terminée")
```

Exercice 2 · Courir avec Maj

Input.is_action_pressed() renvoie true tant que la touche est enfoncée.

```
extends CharacterBody2D

@export var vitesse := 200.0

func _physics_process(_delta):
    var direction := Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
    var v := vitesse * 2 if Input.is_action_pressed("courir") else vitesse
    velocity = direction * v
    move_and_slide()
```

Exercice 3 · Le minuteur de 60 secondes

time_left donne le temps restant ; ceili() l'arrondit à la seconde supérieure, en nombre entier. set_physics_process(false) arrête la fonction de déplacement du joueur.

```
extends Node2D

@onready var minuteur: Timer = $Timer
@onready var affichage: Label = $Label
@onready var joueur = $Joueur

func _process(_delta):
    if not minuteur.is_stopped():
        affichage.text = str(ceili(minuteur.time_left)) + " s"

func _on_timer_timeout():
    affichage.text = "Temps écoulé !"
    joueur.set_physics_process(false)
```

Exercice 4 · Un ennemi qui patrouille

On mémorise la position de départ, on avance dans une direction, et on inverse la direction quand l'écart dépasse 200 pixels. La fonction perdre_une_vie() doit exister dans le script du joueur.

```
extends Area2D

@export var vitesse := 120.0
var depart: float
var sens := 1

func _ready():
    depart = position.x

func _process(delta):
    position.x += sens * vitesse * delta
    if abs(position.x - depart) > 200:
        sens = -sens

func _on_body_entered(body):
    if body.has_method("perdre_une_vie"):
        body.perdre_une_vie()
```